LAPORAN TUGAS BESAR

Milestone 1: Sistem Terdistribusi Dasar

Platform Koordinasi Kebencanaan BNPB

IF4031 Arsitektur Aplikasi Terdistribusi

Semester I Tahun Akademik 2026/2027

Kelompok bebasss

Disusun oleh

Rafif Farras 13523095
Azfa Radhiyya Hakim 13523115
Samantha Laqueenna Ginting 13523138

Program Studi Teknik Informatika

Sekolah Teknik Elektro dan Informatika

Institut Teknologi Bandung

Pernyataan Integritas Akademik dan Penggunaan AI

Kami, anggota kelompok bebasss, menyatakan bahwa laporan dan implementasi Milestone 1 ini merupakan hasil kerja kelompok kami sendiri. Dalam pengerjaannya, kelompok kami menggunakan asisten AI sebagai alat bantu untuk analisis dan debugging kode, penyusunan draf dokumentasi, serta sebagian penulisan laporan ini, dengan seluruh arahan, keputusan desain, dan tinjauan akhir tetap berada di tangan anggota kelompok. Rincian penggunaan AI per lingkup pekerjaan ditunjukkan pada tabel berikut, selaras dengan isian Form Penggunaan AI yang wajib dilampirkan pada pengumpulan sesuai ketentuan mata kuliah.

No	Lingkup Pekerjaan	Digunakan	Tingkat Penggunaan
1	Pemeriksaan Ejaan	Tidak	-
2	Pembuatan Teks	Ya	Sedang – menyusun draf sebagian bagian laporan berdasarkan arahan dan data teknis yang sudah ada; seluruh draf ditinjau dan disetujui anggota sebelum dipakai.
3	Bantuan Diskusi dalam Penyusunan Konten	Ya	Sedang – digunakan untuk diskusi desain, analisis trade-off, dan riset referensi teknis sepanjang pengerjaan; keputusan akhir tetap pada anggota kelompok.
4	Pembuatan Gambar, Video, dan/atau Grafik	Tidak	-
5	Lainnya – bantuan analisis dan penulisan kode (implementasi, refactor, unit test, debugging lintas-service)	Ya	Sedang – arahan, keputusan arsitektur/desain, dan tinjauan akhir seluruh kode tetap dilakukan oleh anggota kelompok.

Rafif Farras (13523095)	Azfa Radhiyya Hakim (13523115)	Samantha Laqueenna Ginting (13523138)

Pembagian Tugas

Anggota	Kontribusi
Rafif Farras (13523095)	Problem 3 (Service-to-Service Authentication & Trust) dan Problem 5 (Fan-out Event Distribution & Pub-sub): Auth Service (penerbitan dan pembaruan access/refresh token), RBAC field filtering di Client-Facing API, topologi *fanout* RabbitMQ beserta consumer `dashboard-updater` dan `notifier`. Juga menangani perbaikan lintas-service (polling per sumber, validasi API internal, pagination/limit Canonical Store, refactor `bmkg-mock` dan `pvmbg-mock` menjadi testable) serta penulisan laporan Problem 3 dan Problem 5.
Azfa Radhiyya Hakim (13523115)	Problem 2 (Concurrency, Availability, & Graceful Degradation) dan Problem 4 (Microservice Architecture & Flexible Storage): fondasi Aggregator (polling, circuit breaker, pembatasan konkurensi panggilan PVMBG), Canonical Store PostgreSQL, integrasi Client-Facing API ke API internal Aggregator, serta skenario *load test* k6. Menulis laporan Problem 2 lengkap dengan hasil pengukuran k6.
Samantha Laqueenna Ginting (13523138)	Problem 1 (Arsitektur & Interoperabilitas Data): `bmkg-mock` dan `pvmbg-mock`, *shared types*, serta modul *mapping* yang mengubah data sumber menjadi *HazardEvent* kanonik. Turut berdiskusi untuk batas service pada Problem 4 dan menulis laporan Problem 4.

Daftar Isi

Pernyataan Integritas Akademik dan Penggunaan AI

Pembagian Tugas

1 Pendahuluan **1**

1.1 Latar Belakang . . . 1
1.2 Tujuan . . . 1
1.3 Ruang Lingkup Milestone 1 . . . 1
1.4 Batasan Sistem . . . 2
1.5 Asumsi . . . 2
1.6 Petunjuk Menjalankan Sistem . . . 3
1.7 Struktur Laporan . . . 4

2 Arsitektur Sistem **4**

2.1 Gambaran Umum . . . 4
2.2 Komponen dan Batas Service . . . 6
2.3 Alur Komunikasi dan Data . . . 6
2.4 Teknologi dan Konfigurasi . . . 6

3 Problem 1: Arsitektur dan Interoperabilitas Data **8**

3.1 Rancangan Solusi . . . 8
3.1.1 Aturan Pemetaan ke HazardEvent . . . 8
3.1.2 Kebijakan Evolusi Skema . . . 9
3.2 Alternatif dan Trade-off . . . 9
3.3 Proof of Concept (PoC) . . . 10

4 Problem 2: Concurrency, Availability, dan Graceful Degradation **12**

4.1 Rancangan Solusi . . . 12
4.1.1 Tiga Risiko yang Ditangani . . . 13
4.1.2 Model Konkurensi dan Isolasi Sumber . . . 13
4.1.3 Graceful Degradation dan Data Stale . . . 13
4.2 Alternatif dan Trade-off . . . 15
4.3 Proof of Concept (PoC) . . . 15

4.3.1 Uji Latensi BMKG-only 15
4.3.2 Uji Beban Berkelanjutan 16
4.3.3 Outage dan Pemulihan PVMBG 17

5 Problem 3: Service-to-Service Authentication dan Trust 19
5.1 Rancangan Solusi 19
5.2 Alternatif dan Trade-off 20
5.3 Batasan yang Disadari 21
5.4 Bukti Penyelesaian 21
5.5 Jawaban atas Butir "Di Laporan" 22

6 Problem 4: Arsitektur Microservice dan Flexible Storage 23
6.1 Rancangan Solusi 23
6.2 Alternatif dan Trade-off 23
6.3 Batasan yang Disadari 24
6.4 Bukti Penyelesaian 24

7 Problem 5: Fan-out Event Distribution dan Pub-sub 28
7.1 Rancangan Solusi 28
7.2 Alternatif dan Trade-off 29
7.3 Batasan yang Disadari 29
7.4 Bukti Penyelesaian 30
7.5 Jawaban atas Butir "Di Laporan" 31

A Bukti Pengujian Tambahan 31

1 Pendahuluan

1.1 Latar Belakang

Penanggulangan bencana melibatkan lembaga-lembaga yang bekerja secara otonom, memiliki sumber data, format informasi, dan karakteristik infrastruktur yang berbeda. Dalam skenario tugas ini, BMKG menyediakan informasi kejadian seismik dan potensi tsunami, sedangkan PVMBG menyediakan laporan aktivitas vulkanik. BNPB membutuhkan kedua aliran informasi tersebut untuk membangun gambaran situasi yang terpadu dan menyajikannya kepada pihak yang berwenang. Namun, perbedaan format data, latensi layanan, dan batas kepercayaan antarlembaga membuat integrasi langsung menjadi tidak memadai.

Masalah tersebut tidak hanya berkaitan dengan pertukaran data. Ketika satu layanan sumber melambat atau gagal, layanan lain tetap dapat menghasilkan data yang bernilai bagi pengguna. Demikian pula, lonjakan permintaan pada saat status bencana meningkat tidak boleh membuat seluruh sistem berhenti merespons. Sistem juga perlu membedakan hak akses antara Media, Tim Lapangan, dan BNPB Internal agar data mentah seperti koordinat presisi serta atribut spesifik sumber tidak bocor kepada pihak yang hanya membutuhkan ringkasan.

Oleh sebab itu, proyek ini membangun proof-of-concept platform koordinasi kebencanaan BNPB sebagai aplikasi terdistribusi. Platform menyatukan data heterogen dari BMKG dan PVMBG melalui komunikasi jaringan, mengubahnya ke format kanonik *HazardEvent*, menyimpannya pada *Canonical Store*, dan mendistribusikan informasi tersebut melalui API serta mekanisme *publish-subscribe*.

1.2 Tujuan

Tujuan utama pengembangan sistem adalah membuktikan bahwa beberapa layanan otonom dapat diintegrasikan menjadi satu platform koordinasi tanpa menghilangkan batas kepemilikan dan karakteristik masing-masing layanan. Secara khusus, Milestone 1 bertujuan untuk:

1. menyatukan data seismik dan vulkanik dari dua instansi ke dalam format kanonik yang konsisten serta tetap menerima perubahan skema aditif;
2. menjaga ketersediaan data penting melalui pemrosesan asinkron, pembatasan konkurensi, dan degradasi layanan yang eksplisit ketika salah satu sumber tidak tersedia;
3. menerapkan pemisahan domain kepercayaan untuk komunikasi antarlayanan dan akses client downstream;
4. memisahkan komponen ke dalam layanan dan container yang dapat dijalankan secara independen, dengan penyimpanan yang mendukung atribut dinamis; serta
5. mendistribusikan event baru ke beberapa consumer melalui message broker tanpa membuat Aggregator bergantung langsung pada consumer tersebut.

1.3 Ruang Lingkup Milestone 1

Ruang lingkup implementasi dibatasi pada dua instansi sumber dan dua jenis bencana. BMKG disimulasikan sebagai sumber *SeismicEvent* dan *TsunamiWarning*; PVMBG disimulasikan

sebagai sumber *VolcanicReport*. Kedua sumber bersifat pasif dan hanya merespons polling berkala dari Aggregator BNPB. Data dari sumber tidak dibagikan melalui pemanggilan fungsi langsung atau library *business logic* bersama, melainkan melalui API jaringan sesuai kontrak layanan.

Komponen BNPB dalam Milestone 1 meliputi Aggregator atau gateway, Canonical Store, Client-Facing API, Auth/Token Service, message broker, dan sedikitnya dua event consumer. Aggregator melakukan polling, pemetaan, penyimpanan, serta publikasi event baru. Canonical Store hanya dimiliki dan diakses langsung oleh Aggregator; layanan lain memperoleh data melalui API Aggregator. Client-Facing API bersifat *read-only* dan menerapkan hak akses berdasarkan scope token.

Hak akses client dibagi menjadi tiga tingkat. Media hanya menerima field berklasifikasi Ringkasan. Tim Lapangan memperoleh data lengkap untuk kebutuhan operasional, sedangkan BNPB Internal memiliki hak akses tertinggi. Access token bersifat berumur pendek dan harus dapat diperbarui melalui alur *refresh* tanpa login ulang. Batasan ini dibuat agar keamanan, ketahanan, dan distribusi data dapat diuji secara nyata pada arsitektur yang sama.

1.4 Batasan Sistem

Milestone 1 belum mencakup seluruh kebutuhan sistem kebencanaan nasional. Sistem tidak menangani jalur pelaporan dari lapangan ke BNPB, perubahan data oleh client downstream, maupun sumber bencana lain seperti kebakaran hutan dan hidrometeorologi. Fokusnya adalah fondasi interoperabilitas, availability, keamanan, fleksibilitas storage, dan distribusi event pada skala dasar.

Pengembangan untuk Milestone 2 berada di luar cakupan laporan ini. Tahap berikutnya akan menguji fondasi yang sama dalam kondisi yang lebih skalabel, termasuk banyak instance layanan, volume data yang meningkat drastis, dan kegagalan beberapa komponen secara bersamaan.

1.5 Asumsi

Beberapa keputusan perancangan dan implementasi berikut diambil sebagai asumsi yang disadari, bukan kelalaian, dan terdokumentasi di sini agar jelas batasannya untuk Milestone 2.

- **Overlap lookback tetap 60 detik, dihitung dari sinkronisasi sukses terakhir.** Polling pertama suatu sumber tidak memakai parameter `since` (mengambil seluruh data tersedia). Polling berikutnya memakai `since` = waktu sinkronisasi sukses terakhir dikurangi `POLL_LOOKBACK_MS` (default 60000 ms), bukan `now()` yang lepas dari riwayat sinkronisasi. Overlap ini membuat event di sekitar batas waktu polling sebelumnya diproses ulang secara aman (upsert berdasarkan `source` dan `source_ref_id`). Asumsinya adalah selisih waktu commit/replikasi di sisi sumber tidak akan melampaui jendela overlap ini pada skala Milestone 1; overlap yang lebih pendek berisiko melewatkan data yang baru tampak setelah polling sebelumnya selesai, sedangkan overlap yang lebih panjang menambah beban query tanpa manfaat berarti pada volume data saat ini.

- **Deduplikasi consumer bersifat in-memory, bukan persisten.** `dashboard-updater` dan `notifier` menolak `hazard_id` yang sudah pernah diproses menggunakan struktur `SeenSet` berkapasitas terbatas di memori proses (lihat `src/dedup.ts` pada masing-masing consumer). Asumsinya adalah consumer berjalan sebagai instance tunggal yang tidak restart terlalu sering; riwayat dedup hilang saat proses direstart, sehingga event yang di-redeliver oleh broker tepat setelah restart dapat diproses ulang. Ini dianggap dapat diterima karena efek samping consumer pada Milestone 1 bersifat simulasi (log), bukan transaksi yang tidak boleh terulang.
- **Access token tidak memiliki mekanisme blocklist/revocation.** Token yang sudah diterbitkan oleh `auth-service` tetap valid hingga kedaluwarsa meskipun client dianggap perlu dicabut aksesnya lebih awal. Asumsinya adalah umur token yang pendek (default 60 detik untuk access token) sudah memadai sebagai mitigasi risiko pada skala proof-of-concept ini.
- **`hazard_id` dibangkitkan oleh Canonical Store, bukan oleh sumber data.** BMKG dan PVMBG masing-masing memiliki pengenal sendiri (`source_ref_id`); `hazard_id` kanonik dibangkitkan sekali oleh Aggregator saat baris pertama kali disisipkan dan dipertahankan pada setiap pembaruan berikutnya (`ON CONFLICT` pada pasangan `source`, `source_ref_id`). Asumsinya adalah pasangan tersebut cukup unik untuk mengidentifikasi satu kejadian sepanjang hidup Milestone 1.
- **Setiap layanan berjalan sebagai instance tunggal.** Tidak ada layanan yang di-scale ke lebih dari satu container pada `docker-compose.yml`. Konsekuensinya, pembatasan konkurensi (misalnya limiter panggilan PVMBG) dan dedup consumer bersifat lokal per proses, bukan terkoordinasi lintas instance. Multi-instance secara eksplisit berada di luar cakupan Milestone 1 dan menjadi perhatian Milestone 2.

1.6 Petunjuk Menjalankan Sistem

Langkah lengkap menjalankan sistem, termasuk daftar variabel konfigurasi dan cara memicu setiap skenario demo per problem, didokumentasikan di `README.md` pada root repository. Ringkasannya:

1. Salin `.env.example` menjadi `.env`, lalu ganti seluruh nilai `replace-with-...` dengan kredensial lokal.
2. Bangkitkan token PVMBG dengan `node scripts/mint-pvmbg-token.js` dan simpan hasilnya ke `PVMBG_ACCESS_TOKEN` pada `.env`. Aggregator tidak akan start tanpa token ini.
3. Jalankan seluruh sistem dengan `docker compose up -d -build`.
4. Pastikan setiap service sehat melalui endpoint `/health` masing-masing (BMKG `8081`, PVMBG `8082`, Client API `8080`, Aggregator `8090`, Auth Service `8091`, Dashboard Consumer `8092`, Notifier `8093`).

README juga memuat tabel variabel konfigurasi per kelompok layanan, serta perintah `curl`/`bash` (dan padanan PowerShell) untuk memicu setiap skenario demo Problem 1 sampai Problem 5, termasuk skenario outage, RBAC, dan load test k6.

1.7 Struktur Laporan

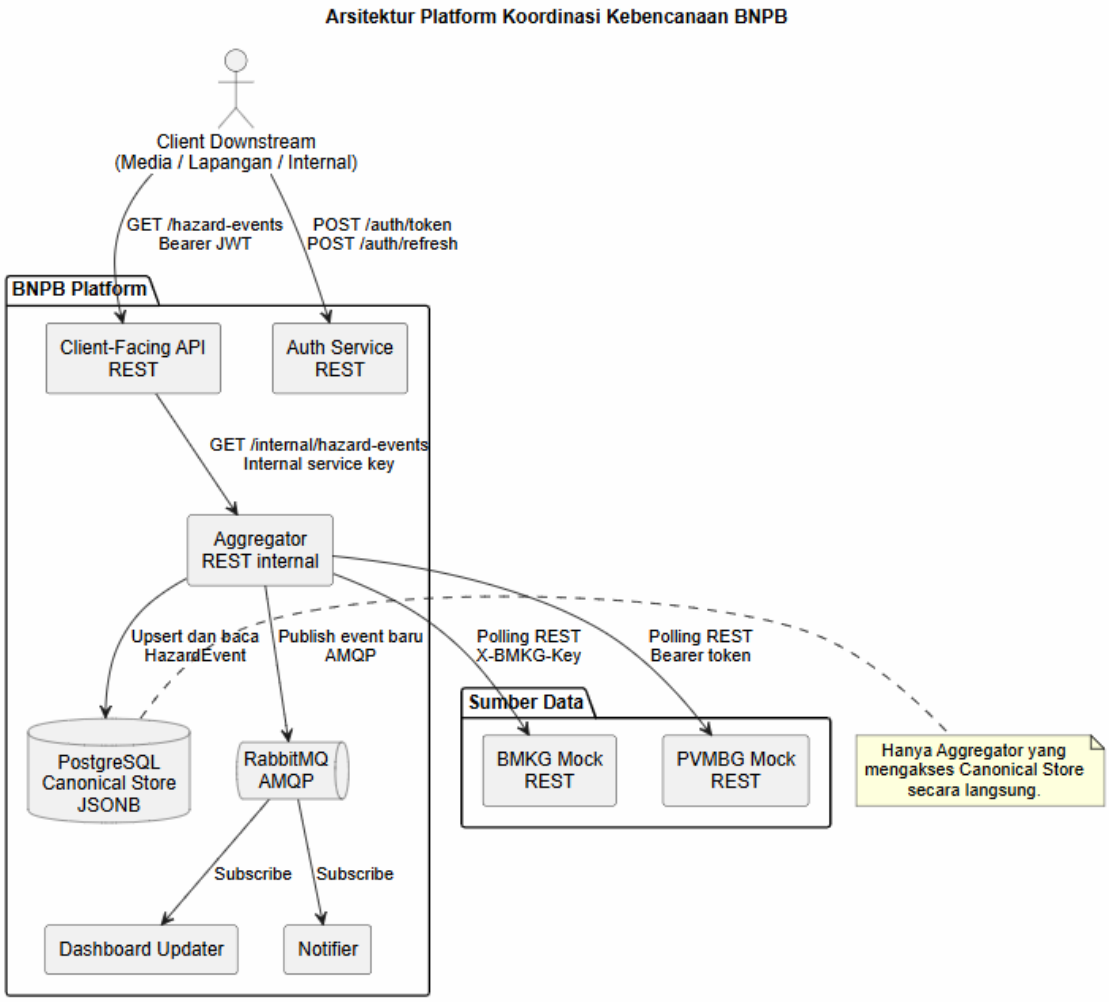
Bab berikutnya menjelaskan arsitektur sistem, batas tiap service, serta teknologi yang digunakan. Setelah itu, setiap problem resmi Milestone 1 dibahas secara terpisah, mencakup rancangan solusi, alternatif dan trade-off, batasan yang disadari, serta bukti penyelesaian. Lampiran digunakan untuk menyimpan bukti pengujian tambahan seperti tangkapan layar, log ber-correlation ID, dan hasil pengukuran.

2 Arsitektur Sistem

2.1 Gambaran Umum

Sistem dibangun sebagai sekumpulan service yang dikemas dan dijalankan terpisah melalui Docker Compose. BMKG dan PVMBG berperan sebagai *mock service*. Aggregator melakukan polling ke kedua sumber, memetakan respons ke format kanonik *HazardEvent*, menyimpannya pada Canonical Store, dan menerbitkan event baru ke message broker.

Client downstream tidak membaca database secara langsung. Permintaan data masuk melalui Client-Facing API, yang memverifikasi token client, menerapkan filter hak akses, lalu mengambil data dari API internal milik Aggregator. Event baru diterima consumer independen untuk kebutuhan dashboard dan notifikasi. Diagram arsitektur sistem ditunjukkan pada Gambar 1.



Gambar 1: Arsitektur platform koordinasi kebencanaan BNPB.

2.2 Komponen dan Batas Service

Tabel 1: Komponen utama sistem.

Komponen	**Antarmuka**	**Tanggung Jawab**
BMKG Mock	REST	Menyediakan data *SeismicEvent* dan *TsunamiWarning*; akses menggunakan `X-BMKG-Key`.
PVMBG Mock	REST	Menyediakan *VolcanicReport*, perubahan skema, dan simulasi outage; akses menggunakan *Bearer token*.
Aggregator	REST internal	Polling, mapping, penyimpanan Canonical Store, serta publikasi event baru.
PostgreSQL	Koneksi database internal	Menyimpan *HazardEvent* dengan atribut dinamis pada kolom `JSONB`; hanya Aggregator yang mengaksesnya langsung.
Client-Facing API	REST	Menyajikan data untuk client, memvalidasi JWT, dan menerapkan RBAC.
Auth Service	REST	Menerbitkan dan memperbarui token client.
RabbitMQ	AMQP	Broker distribusi event dengan pola fan-out.
Event Consumer	AMQP consumer	Menerima event baru secara independen untuk dashboard dan notifikasi.

2.3 Alur Komunikasi dan Data

Aggregator melakukan polling berkala melalui REST. Kredensial untuk BMKG berupa API key, sedangkan PVMBG menggunakan *Bearer token*. Respons sumber dipetakan ke *HazardEvent* dan disimpan oleh Aggregator ke PostgreSQL. Field dinamis disimpan pada `attributes` bertipe `JSONB`.

Setiap event baru yang tersimpan diterbitkan Aggregator ke RabbitMQ. Dashboard Updater dan Notifier berlangganan melalui broker; Aggregator tidak memanggil consumer secara langsung. Untuk alur baca, client memperoleh access token dari Auth Service dan mengirimkannya ke Client-Facing API. Setelah memvalidasi token dan scope, Client-Facing API memanggil endpoint internal Aggregator dengan *internal service key*. Dengan demikian, PostgreSQL bukan dependency langsung bagi Client-Facing API maupun consumer.

2.4 Teknologi dan Konfigurasi

Service aplikasi dibangun menggunakan Node.js, TypeScript, dan Fastify. Komunikasi sinkron menggunakan REST, sedangkan distribusi event memakai RabbitMQ melalui AMQP. Canonical Store menggunakan PostgreSQL dengan `JSONB` untuk menyimpan atribut sumber yang dinamis tanpa penambahan kolom baru. Tabel 2 merinci seluruh teknologi yang dipakai beserta alasan pemilihannya.

Tabel 2: Teknologi yang digunakan dan alasan pemilihan.

Teknologi	**Alasan Pemilihan**
TypeScript (strict mode)	Tipe statis menangkap ketidakcocokan kontrak antar-service saat compile, penting karena tiap domain kepercayaan (BMKG, PVMBG, client) punya bentuk payload berbeda.
Fastify	Overhead rendah dan validasi skema bawaan, cocok untuk service kecil yang perlu start cepat dan banyak dijalankan sebagai container terpisah.
`pg`	Driver PostgreSQL resmi tanpa lapisan ORM, dipilih agar kueri SQL ke Canonical Store (termasuk `UPSERT` dan filter `JSONB`) tetap eksplisit dan mudah diaudit.
PostgreSQL + `JSONB`	Mendukung evolusi skema aditif (field baru dari sumber langsung masuk ke `attributes`) tanpa migrasi, sesuai kebutuhan Problem 4.
RabbitMQ + `amqplib`	Exchange bertipe *fanout* memenuhi kebutuhan Problem 5: Aggregator menerbitkan event sekali, seluruh consumer menerimanya secara independen tanpa Aggregator mengetahui siapa saja consumer-nya.
`jsonwebtoken`	Implementasi JWT standar untuk menerbitkan dan memvalidasi access/refresh token client serta domain kepercayaan PVMBG, tanpa menulis ulang logika signing/verifikasi sendiri.
`zod`	Validasi skema environment variable saat startup (`config.ts` tiap service), sehingga konfigurasi yang salah membuat service gagal start lebih awal alih-alih gagal diam-diam saat runtime.
`pino`	Logger terstruktur (JSON) dengan dukungan `correlation_id` lewat `AsyncLocalStorage`, dipilih agar log antar-service bisa ditelusuri dalam satu alur permintaan.
`vitest`	Test runner cepat dan kompatibel langsung dengan TypeScript/ESM tanpa konfigurasi transpiler tambahan, dipakai untuk 131 unit test di seluruh service.
k6	Load testing terskrip untuk mengukur throughput dan latensi p95 pada Problem 2, dengan dukungan skenario VU paralel yang meniru beban BMKG-only dan beban campuran.
Docker Compose	Orkestrasi lokal yang mengemas setiap service sebagai container terpisah dengan jaringan internal sendiri, selaras dengan batas service yang dituntut Problem 4.

Seluruh service dijalankan melalui `docker-compose.yml`. Port, URL service, credential, interval polling, parameter circuit breaker, koneksi database, koneksi broker, dan kunci internal dikonfigurasi melalui environment variable.

3 Problem 1: Arsitektur dan Interoperabilitas Data

Pertanyaan Utama

Bagaimana Aggregator BNPB dapat menyatukan data BMKG dan PVMBG ke format kanonik yang tetap valid ketika PVMBG menambahkan field baru tanpa restart atau redeploy?

3.1 Rancangan Solusi

Sistem menggunakan pola integrasi *pull* melalui REST. Aggregator melakukan polling ke BMKG untuk data *SeismicEvent* dan *TsunamiWarning*, serta ke PVMBG untuk data *VolcanicReport*. Setiap respons dipetakan menjadi kontrak internal tunggal, yaitu *HazardEvent*, sebelum disimpan pada Canonical Store PostgreSQL dan didistribusikan sebagai event. Dengan demikian, service downstream tidak perlu memahami bentuk data khusus milik masing-masing instansi.

3.1.1 Aturan Pemetaan ke HazardEvent

Tabel 3 merangkum aturan pemetaan yang diterapkan oleh Aggregator. Field inti yang diperlukan oleh platform ditempatkan pada kolom kanonik, sedangkan detail sumber ditempatkan pada `attributes`.

Sumber	Data masukan	Pemetaan ke *HazardEvent*
BMKG	*SeismicEvent* dan *TsunamiWarning* terkait	`source=BMKG`, `source_ref_id=event_id`, `hazard_type=SEISMIC`. Nama wilayah dan koordinat diambil dari data gempa. Tingkat keparahan mengikuti peringatan tsunami bila tersedia; jika tidak tersedia, magnitude $\geq$ 6,5 dipetakan menjadi `SIAGA`, magnitude $\geq$ 5,0 menjadi `WASPADA`, dan sisanya `NORMAL`.
PVMBG	*VolcanicReport*	`source=PVMBG`, `source_ref_id=report_id`, `hazard_type=VOLCANIC`. `alert_level` dipetakan ke severity `NORMAL`, `WASPADA`, `SIAGA`, atau `AWAS`. Nama serta koordinat gunung diperoleh dari tabel referensi gunung api milik Aggregator.

Tabel 3: Aturan pemetaan data sumber ke format kanonik.

3.1.2 Kebijakan Evolusi Skema

Mekanisme yang dipilih adalah JSON dengan kebijakan *additive-only*. Field inti yang sudah dikenal dipetakan secara eksplisit untuk menentukan identitas hazard, jenis hazard, tingkat keparahan, lokasi, dan waktu kejadian. Setiap field tambahan dari sumber yang bukan field inti diteruskan tanpa transformasi ke objek `attributes`. Objek ini disimpan pada kolom `JSONB` di Canonical Store. Oleh karena itu, penambahan field seperti `confidence_level` oleh PVMBG tidak memerlukan perubahan kolom tabel, restart, maupun redeploy service.

Kebijakan penanganan field tak dikenal yang dideklarasikan pada sistem ini adalah **meneruskan field ke `attributes`**, bukan mengabaikannya. Konsistensi kebijakan tersebut dibuktikan melalui perubahan skema PVMBG; setelah field `confidence_level` diaktifkan, field tersebut tetap tersedia pada `attributes` dari *HazardEvent* hasil Aggregator.

3.2 Alternatif dan Trade-off

Keputusan	Keuntungan	Trade-off dan mitigasi
Kontrak kanonik *HazardEvent*	Client dan consumer cukup memahami satu bentuk data meskipun sumber data heterogen.	Aggregator menjadi tempat pemeliharaan aturan mapping. Aturan dipisahkan pada mapper per sumber dan diuji secara terpisah.
JSONB dengan kebijakan *additive-only*	Field baru dapat diterima tanpa migrasi database ataupun redeploy; informasi sumber tidak hilang.	Validasi field dinamis lebih longgar dan query atribut tertentu lebih kompleks daripada kolom relasional. Field yang dibutuhkan oleh proses inti tetap dipertahankan sebagai field kanonik yang tervalidasi.
Mapping eksplisit untuk field inti, `attributes` untuk sisanya	Semantik inti seperti severity dan lokasi konsisten, sementara evolusi aditif tetap fleksibel.	Perubahan makna atau tipe field inti tidak dapat ditangani otomatis dan perlu perubahan kontrak serta mapper.

Tabel 4: Keputusan desain dan trade-off untuk interoperabilitas data.

Perubahan skema yang didukung secara sadar adalah penambahan field opsional pada objek laporan PVMBG dan BMKG. Perubahan yang tidak didukung tanpa perubahan kode adalah penghapusan, penggantian nama, perubahan tipe, atau perubahan makna dari field inti; misalnya `report_id`, `volcano_id`, `alert_level`, dan `reported_at` pada PVMBG. Sistem juga mengharapkan envelope respons tetap berbentuk `{data: [...]}`. Perubahan yang melanggar kontrak tersebut diperlakukan sebagai perubahan *breaking* dan harus disertai pembaruan mapper serta pengujian kompatibilitas.

3.3 Proof of Concept (PoC)

Langkah berikut digunakan untuk menunjukkan bahwa pemetaan kedua sumber berjalan dan perubahan skema aditif dapat diterima saat sistem aktif.

1. Nyalakan seluruh sistem dan pastikan setiap container sehat.

```
docker compose up -d --build
docker compose ps
```

2. Pastikan PVMBG menggunakan skema awal dengan menonaktifkan field tambahan. Endpoint admin ini hanya dipakai untuk demo/pengujian.

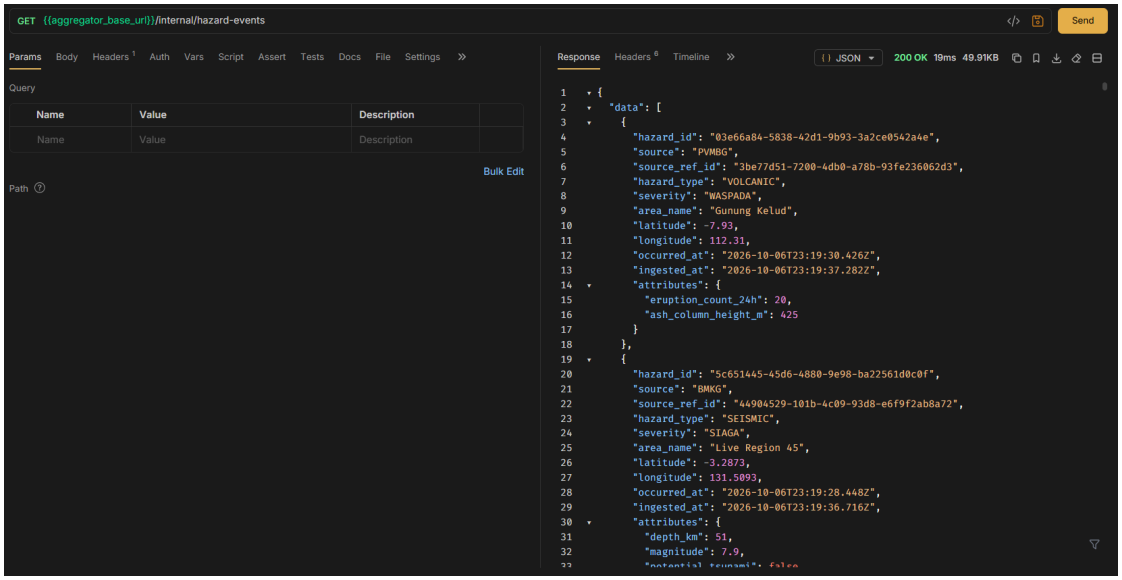
```
Invoke-RestMethod -Method Post `
  -Uri http://localhost:8082/admin/schema-version `
  -ContentType "application/json" `
  -Body '{"enabled": false}'
```



Gambar 2: Status seluruh container setelah menonaktifkan field tambahan PVMBG.

3. Tunggu satu atau dua interval polling, kemudian ambil data kanonik dari API internal Aggregator. Respons harus memperlihatkan hazard dari kedua sumber dengan `source` masing-masing `BMKG` dan `PVMBG`.

```
$headers = @{ "X-Internal-Key" = "<INTERNAL_SERVICE_KEY dari .env>" }
Invoke-RestMethod `
  -Uri "http://localhost:8090/internal/hazard-events" `
  -Headers $headers
```



Gambar 3: Respons API internal Aggregator menampilkan data dari BMKG dan PVMBG.

4. Aktifkan perubahan skema saat semua service tetap berjalan. Tidak ada perintah restart atau redeploy setelah langkah ini.

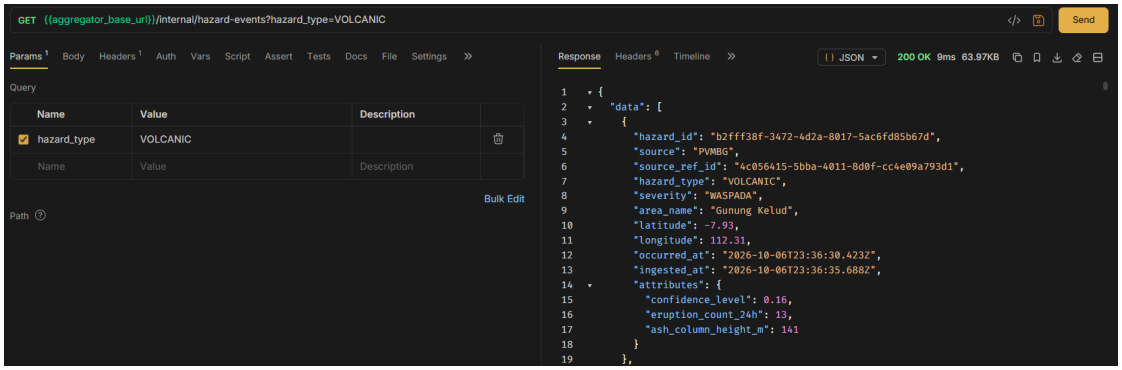
```
Invoke-RestMethod -Method Post `
  -Uri http://localhost:8082/admin/schema-version `
  -ContentType "application/json" `
  -Body '{"enabled": true}'
```



Gambar 4: Mengaktifkan field tambahan PVMBG saat sistem berjalan.

5. Tunggu sedikitnya 15 detik agar PVMBG menghasilkan laporan baru dan Aggregator menyelesaikan polling. Ambil hanya data vulkanik, lalu verifikasi bahwa salah satu objek memiliki `attributes.confidence_level` tanpa ada perubahan schema tabel.

```
Invoke-RestMethod `
  -Uri "http://localhost:8090/internal/hazard-events?hazard_type=VOLCANIC" `
  -Headers $headers
```



Gambar 5: Respons API internal Aggregator setelah perubahan

6. Verifikasi bahwa Aggregator tetap sehat dan tidak mengalami crash.

```
docker compose ps aggregator
docker compose logs --tail=100 aggregator
```



Gambar 6: Perubahan scema version

Dengan proof tersebut, kriteria penyelesaian terpenuhi, yaitu sistem memetakan kedua sumber pada skema awal, menerima perubahan skema aditif ketika sedang berjalan, dan mempertahankan field tambahan secara konsisten di `attributes`.

4 Problem 2: Concurrency, Availability, dan Graceful Degradation

Pertanyaan Utama

Bagaimana sistem tetap responsif saat BMKG dan PVMBG memiliki latensi berbeda, menangani lonjakan permintaan, dan tetap melayani data yang tersedia ketika salah satu sumber mengalami outage?

4.1 Rancangan Solusi

Problem ini memisahkan jalur pembaruan data dari jalur pembacaan client. Aggregator mengambil data BMKG dan PVMBG secara berkala, menyimpannya pada Canonical Store, sedangkan Client API membaca data yang sudah tersedia melalui API internal Aggregator. Dengan pemisahan tersebut, client tidak perlu menunggu panggilan langsung ke sumber eksternal yang lambat pada setiap permintaan `GET /hazard-events`.

4.1.1 Tiga Risiko yang Ditangani

Rancangan ini menangani tiga kegagalan. *Head-of-line blocking* terjadi ketika pengambilan data BMKG dan PVMBG dilakukan sekuensial sehingga sumber cepat tertahan oleh sumber lambat. *Resource exhaustion* terjadi ketika polling tumpang-tindih atau panggilan ke sumber lambat tidak dibatasi. Ketiadaan degradasi terkendali terjadi ketika satu sumber gagal dan seluruh respons client ikut gagal, padahal data terakhir masih tersedia dan berguna.

4.1.2 Model Konkurensi dan Isolasi Sumber

Model konkurensi yang dipilih adalah asynchronous I/O Node.js. BMKG dan PVMBG tidak berbagi satu siklus polling bersama – masing-masing sumber memiliki *interval timer* independen sendiri yang berjalan secara asynchronous dan paralel satu sama lain.

Setiap sumber memiliki guard *in-flight* miliknya sendiri: tick berikutnya untuk sumber yang sama dilewati apabila tick sebelumnya milik sumber itu masih berjalan, tetapi ini tidak memengaruhi sumber lain sama sekali. Dengan demikian, PVMBG yang lambat atau macet tidak pernah menunda ataupun melewatkan tick BMKG – BMKG tetap berjalan sesuai jadwalnya sendiri. Kegagalan satu sumber juga tidak membatalkan hasil sumber lain. Pendekatan per-sumber independen ini mencegah ledakan request ke sumber eksternal ketika PVMBG memiliki latensi tinggi, sekaligus menghilangkan *head-of-line blocking* antar sumber sepenuhnya, bukan hanya mengurangi dampaknya dalam satu siklus bersama.

Setiap panggilan keluar mempunyai timeout yang dikonfigurasi melalui `SOURCE_REQUEST_TIMEOUT_MS` serta mencatat `latency_ms` pada log terstruktur. Untuk PVMBG, client sumber juga dibungkus oleh *circuit breaker* dan *concurrency limiter*. Circuit breaker membuka rangkaian setelah sejumlah kegagalan berturut-turut, menolak request selama masa *cooldown*, lalu mengizinkan satu percobaan pemulihan. Concurrency limiter membatasi jumlah panggilan PVMBG yang aktif sesuai `PVMBG_MAX_CONCURRENT_CALLS`.

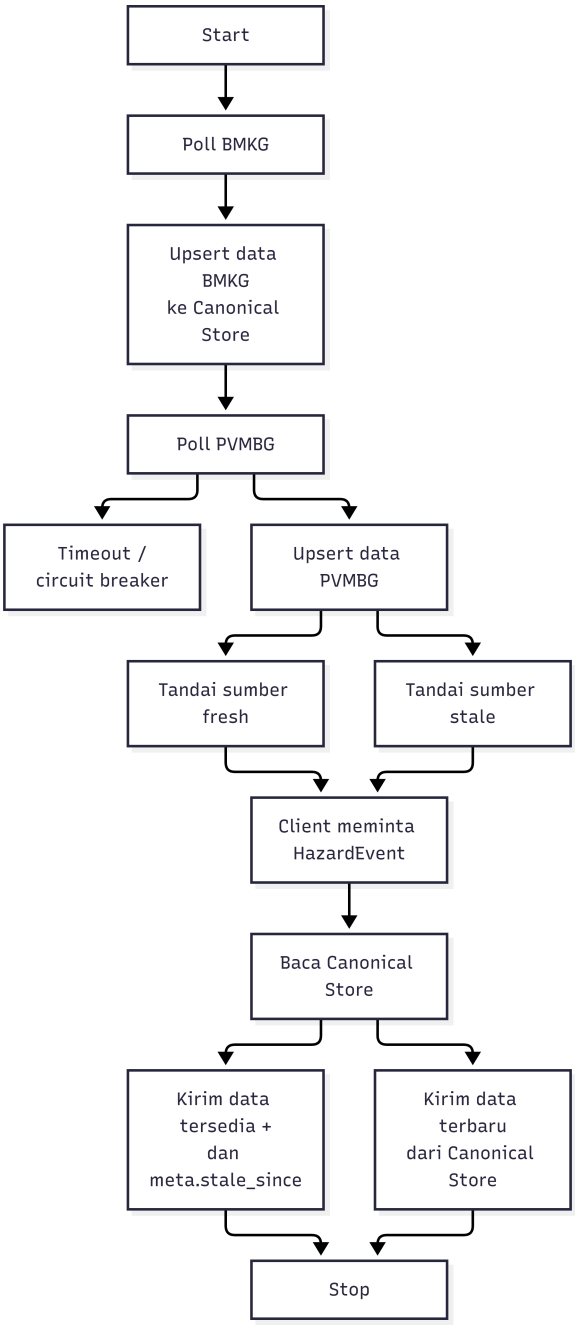
4.1.3 Graceful Degradation dan Data Stale

Jika polling suatu sumber gagal, Aggregator tidak menghapus event yang telah tersimpan. Sebaliknya, sumber tersebut diberi penanda stale pada tabel `source_sync_state`; respons pembacaan memuat `meta.stale_since`. Dengan demikian, client masih memperoleh data terakhir yang diketahui sistem, sekaligus mengetahui bahwa sinkronisasi salah satu sumber sedang bermasalah. Setelah polling sumber kembali berhasil, penanda stale dihapus tanpa restart service.

Untuk mengurangi beban polling ketika histori bertambah, Aggregator menyimpan waktu sinkronisasi sukses terakhir per sumber. Polling pertama mengambil data yang tersedia, sedangkan polling berikutnya memakai parameter `since` dengan overlap yang dapat dikonfigurasi melalui `POLL_LOOKBACK_MS`. Event dalam overlap aman diproses ulang karena penyimpanan menggunakan upsert berdasarkan pasangan `source` dan `source_ref_id`.

Keputusan sistem ketika polling PVMBG berhasil atau gagal dirangkum pada Gambar 7. Diagram tersebut menunjukkan bahwa kegagalan PVMBG tidak membatalkan pembaruan

BMKG maupun pembacaan data yang telah tersedia bagi client.



Gambar 7: Keputusan graceful degradation ketika salah satu sumber data gagal atau melampaui batas waktu.

4.2 Alternatif dan Trade-off

Keputusan	Keuntungan	Trade-off dan mitigasi
Polling independen per sumber (bukan satu siklus bersama)	Latensi atau kemacetan PVMBG tidak pernah menunda maupun melewatkan tick BMKG, karena keduanya punya *interval timer* terpisah.	Tick berikutnya untuk sumber yang sama dilewati jika tick sebelumnya milik sumber itu masih berjalan, agar satu sumber tidak dibanjiri request oleh dirinya sendiri.
Canonical Store sebagai read model	Pembacaan client tidak bergantung pada latensi sumber eksternal dan tetap dapat dilayani saat salah satu sumber outage.	Data bersifat *eventually consistent*; data yang dibaca dapat merupakan data terakhir, bukan data real-time dari sumber. Penanda `stale_since` menyatakan kondisi tersebut secara eksplisit.
Timeout, circuit breaker, dan concurrency limiter pada PVMBG	Kegagalan atau kelambatan berulang dari PVMBG tidak terus-menerus mengonsumsi waktu dan koneksi Aggregator.	Data PVMBG dapat tertunda selama circuit breaker terbuka. Data terakhir tetap disajikan dan polling pemulihan dilakukan setelah cooldown.
Incremental polling dengan overlap	Volume data yang diambil ulang berkurang dibandingkan mengambil seluruh histori pada setiap interval.	Event yang terlambat lebih lama daripada overlap dapat terlewat. Overlap dikonfigurasi dan upsert mencegah data overlap menjadi duplikat.

Tabel 5: Keputusan desain dan trade-off untuk concurrency serta availability.

4.3 Proof of Concept (PoC)

4.3.1 Uji Latensi BMKG-only

Pengujian ini membuktikan bahwa keterlambatan PVMBG tidak menyebabkan *head-of-line blocking* pada request yang hanya membutuhkan data BMKG. Langkah pembuktiannya adalah sebagai berikut.

1. Atur `PVMBG_DELAY_MIN_MS=3000` dan `PVMBG_DELAY_MAX_MS=3000` pada `.env`, kemudian recreate hanya service PVMBG.

```
docker compose up -d --force-recreate pvmbg-mock
```

2. Skrip k6 membaca token client dari variabel environment `CLIENT_JWT` (lihat baris 5 pada skrip k6 di bawah). Ambil token tepat sebelum menjalankan k6, agar token masih berlaku selama durasi uji. `ACCESS_TOKEN_TTL_SECONDS` default (60 detik) sama persis dengan durasi uji sustained pada subbagian berikutnya; naikkan nilainya di `.env` (mis. 300) sebelum menjalankan uji beban berkelanjutan agar token tidak kedaluwarsa di tengah uji.

```
export CLIENT_JWT=$(curl -s -X POST http://localhost:8091/auth/token \
  -H 'Content-Type: application/json' \
  -d '{"client_id":"lapangan-client","client_secret":"<
      AUTH_LAPANGAN_CLIENT_SECRET>"}' \
```

```
  | node -pe 'JSON.parse(require("fs").readFileSync(0)).access_token')
```

3. Jalankan skenario *k6*. Skenario ini mengirim request BMKG-only dan request vulkanik secara bersamaan.

```
k6 run tests/phase6/p2-concurrency.k6.js
```

4. Catat p95 workload BMKG-only. Hasilnya dirangkum pada Tabel 6.

Aspek	Hasil	Kriteria
Kondisi PVMBG	Delay tetap 3.000 ms	Delay PVMBG 3 detik
Workload bersamaan	50 VU BMKG-only dan 10 VU vulkanik	Kedua workload berjalan bersamaan
p95 latensi BMKG-only	231,59 ms	< 300 ms
Error rate BMKG-only	0,00%	$< 1\%$
Status threshold BMKG-only	Lulus	Seluruh threshold lulus

Tabel 6: Hasil uji latensi request BMKG-only ketika PVMBG lambat.

Hasil tersebut menunjukkan bahwa keterlambatan PVMBG tidak menyebabkan *head-of-line blocking* pada pembacaan data seismik.

4.3.2 Uji Beban Berkelanjutan

Tooling yang digunakan adalah *k6* dengan script `tests/phase6/p2-concurrency.k6.js`. Script menjalankan 50 virtual user untuk request BMKG-only dan 10 virtual user untuk request vulkanik selama 60 detik secara sustained. Output pengujian harus melaporkan throughput, p50, p95, p99, dan error rate. Sistem dinyatakan memenuhi kriteria apabila error rate, di luar penolakan terkontrol seperti HTTP 429, berada di bawah 1% dan tidak terjadi crash.

Sama seperti sebelumnya, ambil `CLIENT_JWT` tepat sebelum menjalankan k6 (lihat langkah 2 subbagian sebelumnya), dengan `ACCESS_TOKEN_TTL_SECONDS` dinaikkan agar token tidak kedaluwarsa sebelum uji 60 detik ini selesai.

```
k6 run tests/phase6/p2-concurrency.k6.js
```

Hasil pengujian ditunjukkan pada Tabel 7. Nilai p50 ditampilkan oleh *k6* sebagai `med`. Metrik berlabel HTTP dihitung dari seluruh request pada pengujian.

Metrik	Hasil	Kriteria
Konfigurasi beban	50 VU BMKG-only dan 10 VU vulkanik selama 60 detik	≥50 koneksi paralel selama ≥60 detik
Total request	20.244 request	–
Throughput	336,74 request/detik	–
p50 latensi HTTP keseluruhan	165,95 ms	Dilaporkan
p95 latensi HTTP keseluruhan	231,27 ms	Dilaporkan
p99 latensi HTTP keseluruhan	333,48 ms	Dilaporkan
Error rate	0,00% (0 dari 20.244 request)	< 1%
Check skenario	37.049 berhasil, 0 gagal	Seluruh check berhasil

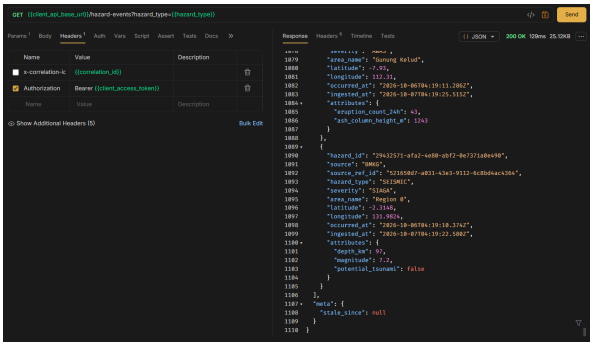
Tabel 7: Hasil load testing sustained menggunakan *k6*.

Dengan error rate 0,00%, pengujian memenuhi batas keandalan yang ditetapkan. Selama pengujian, seluruh service tetap berjalan dan tidak ada penolakan terkontrol HTTP 429 yang tercatat.

4.3.3 Outage dan Pemulihan PVMBG

Pengujian ini membuktikan bahwa kegagalan PVMBG tidak menghentikan jalur pembacaan client dan bahwa sistem pulih tanpa restart service.

1. Kirim request `CLIENT API - /hazard-events` dengan `hazard_type=VOLCANIC`. Terlihat bahwa request berhasil, berisi data vulkanik, dan `meta.stale_since` bernilai `null`.



Gambar 8: Kondisi awal saat service aktif dan `stale_since` bernilai `null`.

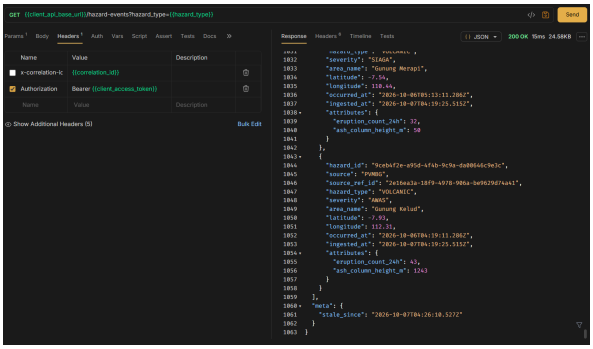
2. Set variable `outage_enabled=true` dan `outage_duration_seconds=30`, lalu jalankan request `PVMBG MOCK - /admin/outage`. Respons menyatakan `outage_active=true`.



Gambar 9: Membuat server down sesaat

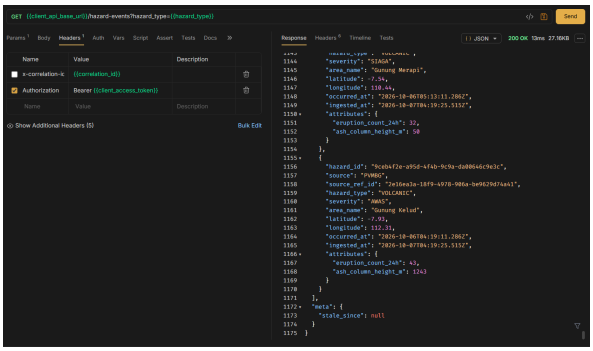
3. Tunggu sekitar 8 detik agar polling yang sedang berlangsung selesai dan Aggregator

menjalankan polling PVMBG yang gagal. Kemudian jalankan kembali request `CLIENT API - /hazard-events` untuk `hazard_type=VOLCANIC`. Respons vulkanik memuat data terakhir dari Canonical Store, sedangkan `meta.stale_since` berisi waktu sejak kapan agregrator mendeteksi kegagalan sinkronisasi.



Gambar 10: `stale_since` terisi saat PVMBG mengalami outage.

4. Setelah durasi outage selesai, tunggu sekitar 8 detik agar satu polling pemulihan berhasil, lalu ulangi request vulkanik. Respons `meta.stale_since` kembali `null`.



Gambar 11: `stale_since` kembali `null` setelah PVMBG pulih.

5. **Bukti tambahan (2026-10-09): BMKG tetap normal selama PVMBG outage.** Skenario di atas membuktikan separuh kriteria (data vulkanik stale dan pulih tanpa restart); separuh lainnya – bahwa data seismik sama sekali tidak terganggu – dibuktikan dengan memicu outage PVMBG 15 detik, lalu langsung mengirim request `hazard_type=SEISMIC` dan `hazard_type=VOLCANIC` ke client-api pada window yang sama. Request SEISMIC tetap `200` dengan `meta.stale_since: null` dan data terbaru (`ingested_at 09:33:05.141Z`), sementara request VOLCANIC pada waktu yang sama sudah menunjukkan `meta.stale_since: "2026-10-09T09:33:02.022Z"` – keduanya dikonfirmasi dari respons API yang sama, bukan sumber berbeda.



Gambar 12: Request SEISMIC (`stale_since: null`) dan VOLCANIC (`stale_since` terisi) pada window outage PVMBG yang sama, 2026-10-09.

5 Problem 3: Service-to-Service Authentication dan Trust

Pertanyaan Utama

Bagaimana setiap domain kepercayaan dipisahkan sehingga kredensial antarsumber tidak dapat dipertukarkan, sementara hak akses client downstream tetap ditegakkan berdasarkan scope dan masa berlaku token?

5.1 Rancangan Solusi

Sistem memiliki tiga domain kepercayaan yang masing-masing memakai format kredensial dan secret yang berbeda:

1. **BMKG**: kunci statis pada header `X-BMKG-Key`, dicocokkan dengan `BMKG_API_KEY` di *bmkg-mock*. Bearer token ditolak dengan 403.
2. **PVMBG**: Bearer JWT HS256 dengan `aud: pvmbg`, ditandatangani `PVMBG_JWT_SECRET`. Token dibuat sekali dengan skrip `scripts/mint-pvmbg-token.js`. Kunci BMKG ditolak dengan 403.
3. **Client downstream**: Bearer JWT HS256 dengan `aud: bnpb-client`, ditandatangani `CLIENT_JWT_SECRET`, diterbitkan oleh *auth-service*. Claim berisi `sub`, `scope` (*media*, *lapangan*, atau *internal*), dan `purpose` (*access* atau *refresh*). TTL access token 60 detik dan refresh token 900 detik, keduanya dapat dikonfigurasi.

Tiga client downstream adalah identitas yang benar-benar berbeda, masing-masing dengan `client_id` dan `client_secret` sendiri dari environment. Perbedaan hak akses ditegakkan di sisi BNPB, bukan oleh flag di sisi client.

Auth-service bersifat stateless. Access dan refresh token adalah JWT yang memverifikasi

dirinya sendiri, sehingga tidak ada tabel token. Refresh memeriksa bahwa client masih terdaftar dengan scope yang sama sebelum menerbitkan pasangan token baru.

Di *client-api*, middleware `auth.ts` memverifikasi JWT dan menyimpan scope pada request. Middleware `rbacFilter.ts` membatasi field: scope *media* hanya menerima field Ringkasan. Permintaan eksplisit `view=raw` dari scope *media* ditolak dengan 403 `FORBIDDEN_FIELD_ACCESS` sebelum panggilan ke Aggregator, sehingga data mentah tidak pernah diambil.

Setiap container hanya menerima variabel yang dibacanya. Tabel 8 merangkum pemetaannya. Sebelumnya seluruh container menerima `.env` penuh.

Service	**Secret yang diterima**
bmkg-mock	`BMKG_API_KEY`
pvmbg-mock	`PVMBG_JWT_SECRET`
aggregator	`BMKG_API_KEY`, `PVMBG_ACCESS_TOKEN`, `INTERNAL_SERVICE_KEY`
client-api	`CLIENT_JWT_SECRET`, `INTERNAL_SERVICE_KEY`
auth-service	`CLIENT_JWT_SECRET`, `AUTH_*_CLIENT_SECRET`
consumers, rabbitmq	`RABBITMQ_URL` atau `RABBITMQ_USER/PASSWORD`

Tabel 8: Pemetaan secret ke service. Client-api tidak menerima kunci BMKG, PVMBG, maupun kredensial auth.

5.2 Alternatif dan Trade-off

Keputusan	Keuntungan	Trade-off dan mitigasi
JWT stateless (dipilih) dibanding token store (Redis atau Postgres)	Tidak ada dependensi tambahan, sesuai kontrak §6.2, dan tidak ada titik kegagalan baru.	Token tidak bisa dicabut sebelum `exp`. Dimitigasi dengan TTL pendek (60 detik).
Satu `CLIENT_JWT_SECRET` untuk semua scope (dipilih) dibanding secret per scope	Konfigurasi sederhana. Ketiga client berada di domain kepercayaan yang sama.	Kebocoran secret memungkinkan pemalsuan token scope mana pun. Dimitigasi dengan membatasi secret hanya ke auth-service dan client-api.
`environment` minimal per service (dipilih) dibanding `env_file` bersama	Kompromi satu container tidak membocorkan secret domain lain.	Daftar variabel per service harus dijaga manual saat ada perubahan.
Kunci statis untuk BMKG (dipilih) dibanding JWT	Sesuai kontrak. Sumber bersifat pasif dan ringan.	Kunci tidak punya masa berlaku dan harus dirotasi manual.
RBAC di client-api (dipilih) dibanding di Aggregator	Aggregator tetap menyimpan data lengkap untuk jalur internal. Filter satu tempat.	Client-api menjadi titik yang harus benar. Dimitigasi dengan unit test untuk filter dan scope.

Tabel 9: Keputusan desain dan trade-off untuk autentikasi dan trust.

5.3 Batasan yang Disadari

1. **Refresh token tidak dicabut setelah dipakai.** Refresh token lama tetap valid sampai `exp` (15 menit). Ini konsekuensi dari auth yang stateless (kontrak §6.2). Jika refresh token bocor, penyerang bisa memperbarui akses hingga 15 menit. Access token lama tetap ditolak setelah TTL 60 detik, sesuai spek.
2. **Penolakan token bergantung pada TTL, bukan blocklist.** Access token yang sudah dicabut secara manual tidak bisa dicabut sebelum kedaluwarsa.
3. **Satu secret untuk semua scope client.** Kompromi `CLIENT_JWT_SECRET` berarti token scope *lapangan* atau *internal* bisa dipalsukan. Pemisahan per scope di luar cakupan M1.
4. **Secret tetap berupa environment variable.** Tidak ada secret manager, dan tidak ada rotasi otomatis. Client secret bersifat statis.
5. **Cakupan M1 hanya baca.** Ketiga client bersifat read-only, sehingga kompromi client belum bisa mengubah data di BNPB.

5.4 Bukti Penyelesaian

Bukti berikut diambil dari pengujian langsung terhadap sistem yang berjalan penuh lewat `docker compose up`, dijalankan 2026-10-07.

No	Skenario	Harapan	Hasil
1	Token *media*, `view=raw`	403	**403** (dikonfirmasi langsung). Kode `FORBIDDEN_FIELD_ACCESS` diverifikasi lewat unit test *client-api* (`app.test.ts`).
2	Token *media*, `GET /hazard-events`	Hanya field Ringkasan	**Sesuai** (dikonfirmasi langsung). Field yang muncul persis 7 field Ringkasan: `area_name`, `hazard_id`, `hazard_type`, `ingested_at`, `occurred_at`, `severity`, `source`. Tidak ada `source_ref_id`, `latitude`, `longitude`, atau `attributes`.
3	Token *lapangan*, tunggu lebih dari 60 detik, lalu gunakan	401	**401** (dikonfirmasi langsung), setelah menunggu 65 detik penuh terhadap TTL default 60 detik.
4	Refresh (`/auth/refresh`), lalu pakai token baru dan token lama	Token baru 200, token lama 401	Token lama: **401** (dikonfirmasi langsung). `POST /auth/refresh` menerbitkan `access_token` baru tanpa resubmit kredensial (dikonfirmasi langsung). Pemakaian token baru ke `/hazard-events` belum di-*re-test* eksplisit pada sesi ini; alur ini tercakup unit test *auth-service*.
5	Kunci BMKG ke PVMBG dan Bearer PVMBG ke BMKG	401 atau 403	`X-BMKG-Key` ke PVMBG: **403** (dikonfirmasi langsung). Kredensial berformat Bearer ke BMKG: **403** (dikonfirmasi langsung memakai token client; BMKG menolak setiap token berformat Bearer, termasuk token PVMBG asli – kasus dengan token PVMBG asli tercatat di *activity log* `2026-10-01_2020`).
6	Cek variabel di container *client-api* (`docker compose exec`)	Tidak memuat secret domain lain	**Sesuai** (dikonfirmasi langsung). Hanya nama variabel `CLIENT_JWT_SECRET` dan `INTERNAL_SERVICE_KEY` yang muncul. `BMKG_API_KEY`, `PVMBG_JWT_SECRET`, `PVMBG_ACCESS_TOKEN`, `AUTH_*_CLIENT_SECRET`, dan `DATABASE_URL` tidak ada.

Tabel 10: Skenario bukti Problem 3, diuji 2026-10-07.

Verifikasi otomatis: 18 unit test *auth-service* dan 28 unit test *client-api* lulus, termasuk filter RBAC, validasi scope, dan validasi parameter query.

5.5 Jawaban atas Butir "Di Laporan"

Mekanisme yang dipakai adalah JWT HS256 stateless dengan tiga trust domain terpisah. Mekanisme ini tidak membuka celah baru karena:

1. setiap domain memakai secret yang berbeda, sehingga token atau kunci satu domain tidak diterima domain lain (dibuktikan dengan skenario 5);

2. `aud` berbeda antara token client (`bnpb-client`) dan token PVMBG (`pvmbg`);
3. `purpose` membedakan access dan refresh, sehingga refresh token tidak bisa dipakai sebagai access token, dan sebaliknya;
4. secret hanya ada di container yang memerlukannya (tabel 8);
5. tidak ada kredensial yang di-hardcode atau di-commit. `.env` di-gitignore dan `.env.example` berisi placeholder.

Celah yang masih ada tercantum pada bagian Batasan.

6 Problem 4: Arsitektur Microservice dan Flexible Storage

Pertanyaan Utama

Bagaimana sistem dipecah menjadi layanan yang dapat dijalankan independen dan menyimpan atribut data yang berubah tanpa migrasi schema atau akses storage lintas service?

6.1 Rancangan Solusi

Untuk mencapai independensi sistem, aplikasi monolitik awal dipecah menjadi beberapa *microservice* (antara lain `aggregator`, `client-api`, dan `auth-service`), di mana setiap layanan dibungkus dalam *container* Docker terpisah via `docker-compose.yml`. Pembagian *boundaries* ditentukan berdasarkan fungsionalitas dan domain: `aggregator` dikhususkan pada beban *ingestion* dari *mock source* serta pengelolaan persistensi, sedangkan `client-api` berfungsi sebagai *API Gateway* untuk mengekspos endpoint ke luar dan memvalidasi akses klien.

Pada sisi penyimpanan (*flexible storage*), kami menggunakan **PostgreSQL**. Alih-alih mendefinisikan kolom statis untuk seluruh detail data, *Canonical Store* menempatkan atribut tambahan dan dinamis pada kolom `attributes` yang bertipe data **JSONB**. Dengan demikian, begitu PVMBG menerbitkan *field* baru seperti `confidence_level`, *field* tersebut langsung tersimpan ke dalam objek JSONB tanpa perlu melakukan proses *schema migration* pada basis data.

Selain itu, kepemilikan basis data sangat dibatasi. Basis data PostgreSQL hanya dapat diakses secara langsung oleh instansi `aggregator`. *Microservice* lain yang membutuhkan data kanonik (seperti `client-api`) wajib memanggil *internal REST API* milik `aggregator`.

6.2 Alternatif dan Trade-off

Kami membandingkan beberapa alternatif arsitektur penyimpanan:

- **Alternatif 1: Document Store murni (contoh: MongoDB)**. *Trade-off*: Menawarkan *schema-less* murni dan sangat tepat untuk dokumen berskema sangat dinamis, namun kami kehilangan dukungan kueri relasional bawaan. Hal ini dapat menghambat fleksibilitas pengembangan kedepan bila ada banyak join relasi struktural antar data internal yang

ketat (misal data sumber vs data notifikasi).

- **Alternatif 2: Basis Data Relasional dengan Skema Konvensional (Kolom Baku)**. *Trade-off*: Paling ketat dalam menjaga validasi tipe data (*type-safe*). Namun, akan sangat menyulitkan operasi berjalan (menyebabkan interupsi sistem saat *deployment* dan migrasi tabel DDL) jika sumber eksternal mengubah *payload* laporan (mengandung kasus Problem 1).
- **Keputusan Akhir (PostgreSQL dengan kolom parsial JSONB)**: Kombinasi terbaik dari dua sisi. *Field* krusial untuk pencarian cepat seperti `hazard_type` dan `occurred_at` dipetakan sebagai kolom relasional yang diindeks, sementara data fleksibel diletakkan pada kolom JSONB `attributes`. Skema *record* campuran lama dan baru dapat berdampingan tanpa migrasi.

6.3 Batasan yang Disadari

Pemisahan *service* ini memperkenalkan tambahan latensi via lapisan jaringan (*HTTP call*) antara `client-api` dan `aggregator`, berbanding monolit konvensional. Penempatan data pencarian ke dalam blok JSONB juga mengurangi kinerja filter data jika kueri dijalankan pada komponen properti yang tertanam lebih dalam bila indeks *GIN* tidak dioptimalkan. Kami menerima kekurangan latensi mikro (*micro-latency*) demi mendapatkan ketahanan sistem terhadap perubahan dan kegagalan (*fault isolation*).

6.4 Bukti Penyelesaian

Kriteria terbukti ketika komponen berdiri independen dalam *container*, dan format JSONB menjaga kompatibilitas sebelum dan sesudah mutasi instansi sumber.

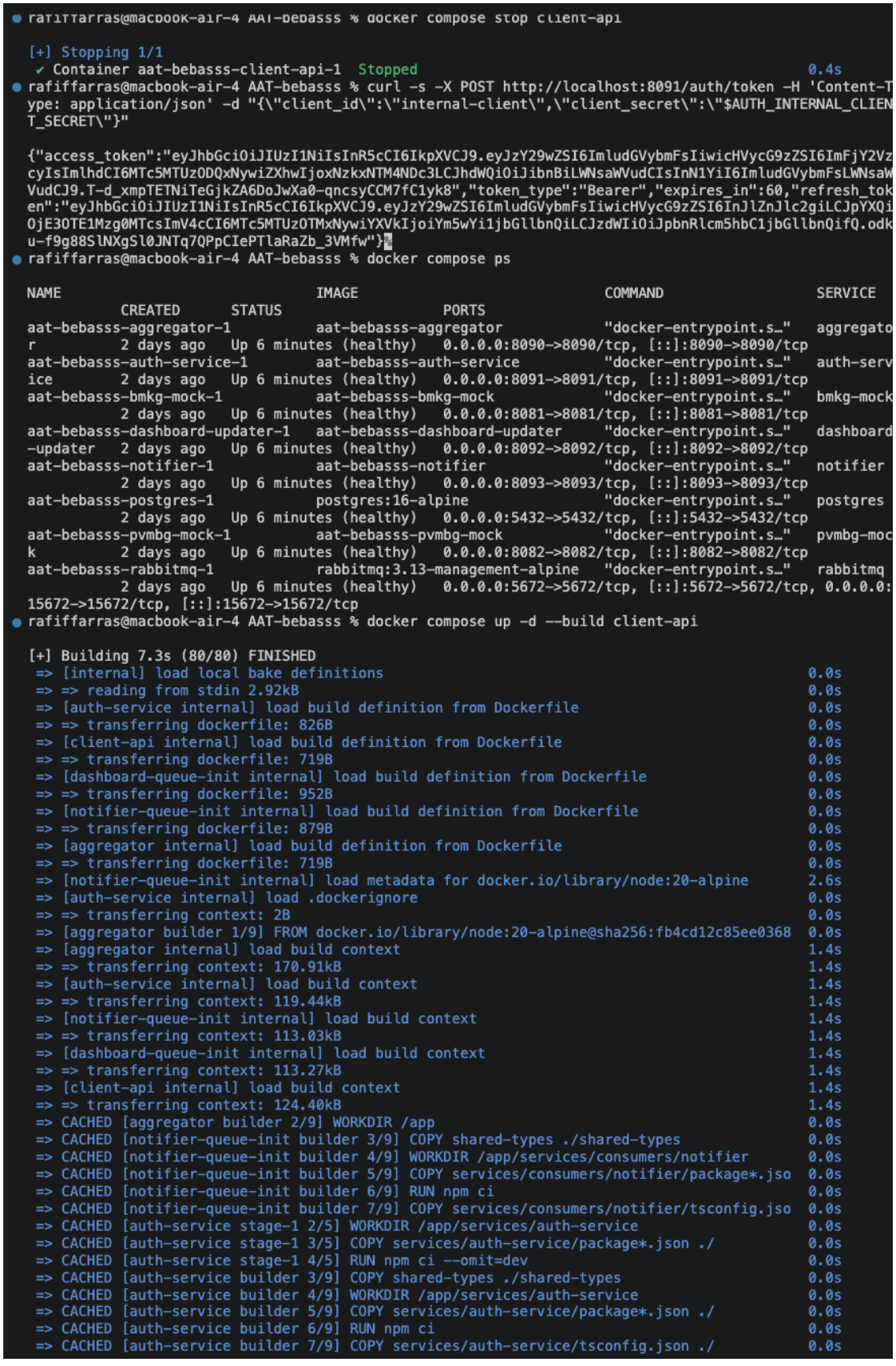
1. Verifikasi bahwa kode menginstruksikan *canonical-store* membuat kolom `attributes` berbasis tipe `JSONB`:

```
CREATE TABLE IF NOT EXISTS hazard_events (
    hazard_id UUID PRIMARY KEY,
    -- (kolom inti konvensional),
    attributes JSONB NOT NULL DEFAULT '{}'::jsonb
);
```

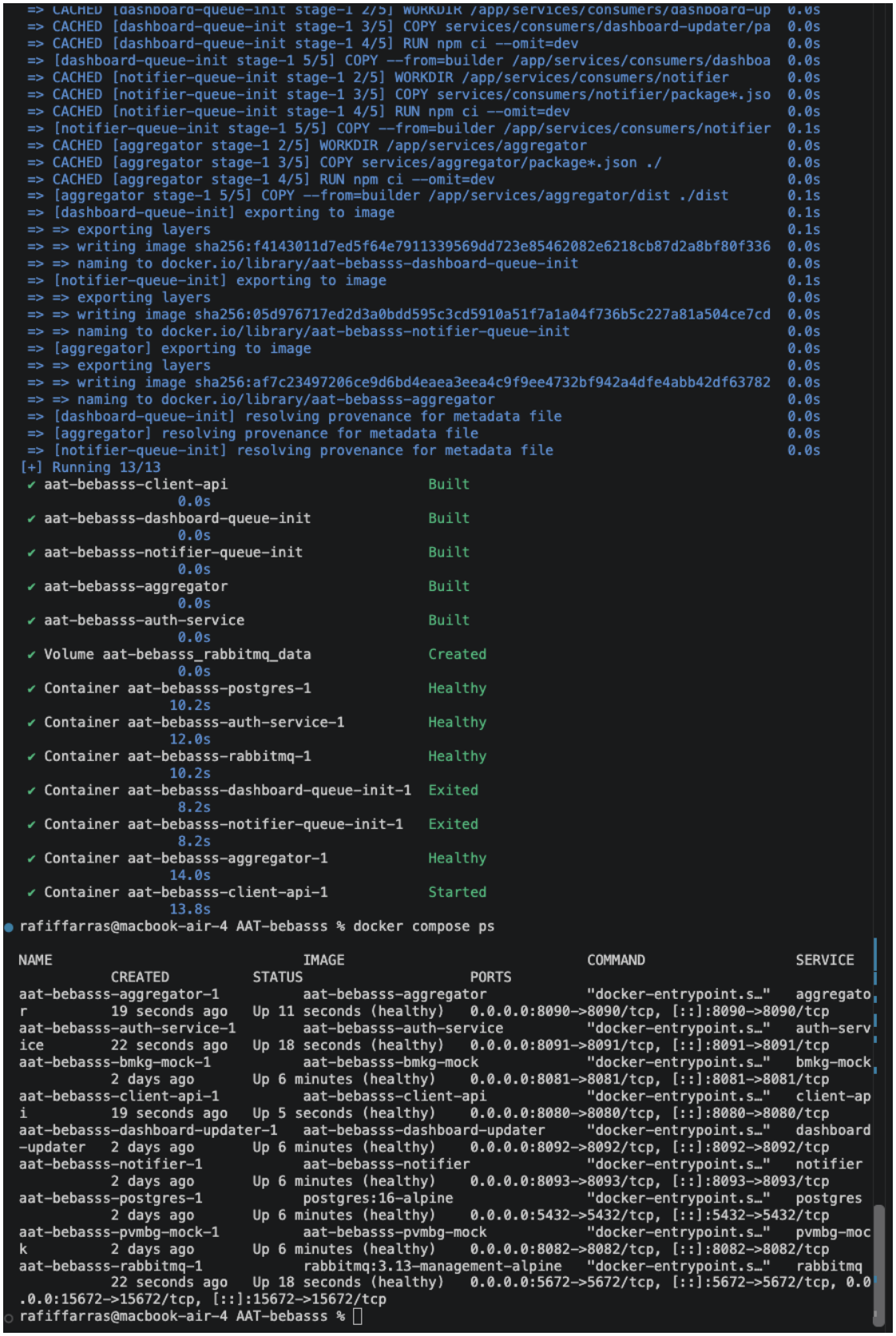
2. Verifikasi independensi *container*: hentikan `client-api` sepenuhnya, *rebuild*, lalu jalankan ulang – sementara itu, buktikan *service* lain yang tidak bergantung padanya (`auth-service`) tetap merespons permintaan normal selama `client-api` mati:

```
docker compose stop client-api
# selagi client-api mati:
curl -s -X POST http://localhost:8091/auth/token \
  -H 'Content-Type: application/json' \
  -d '{"client_id":"internal-client","client_secret":"<
      AUTH_INTERNAL_CLIENT_SECRET>"}'
docker compose ps
docker compose up -d --build client-api
docker compose ps
```

Diuji 2026-10-09: begitu `client-api` dihentikan, `POST /auth/token` ke `auth-service` tetap membalas `200` dengan `access_token` baru – membuktikan layanan lain tidak terinterupsi. `docker compose ps` setelahnya menunjukkan `client-api` tidak lagi berstatus `Up`. Setelah *rebuild* selesai, `client-api` kembali `Up (healthy)` tanpa satu pun *service* lain ikut di-restart.



Gambar 13: `client-api` dihentikan; `auth-service` tetap membalas token baru selagi `client-api` mati; *rebuild* dimulai.



Gambar 14: *Rebuild* selesai; `docker compose ps` menunjukkan `client-api` kembali `Up (healthy)` tanpa service lain ikut terganggu.

3. Verifikasi *co-existence* data lama dan baru (tanpa migrasi) berdampingan dalam JSONB

pada *response payload*. Diuji 2026-10-09: PVMBG direcreate (reset skema ke kondisi lama), ditunggu satu record live baru terbentuk tanpa `confidence_level`, skema diaktifkan, lalu ditunggu satu record live berikutnya yang membawa `confidence_level`. Query `GET /hazard-events?hazard_type=VOLCANIC&limit=2` pada respons yang sama menunjukkan kedua record berdampingan dan sama-sama terbaca – record `d72a6d8b...` tanpa `confidence_level` dan record `95639f2f...` dengan `confidence_level: 0.86` – berjarak hanya 10 detik `occurred_at`, tanpa migrasi kolom apa pun di antaranya.



Gambar 15: Dua record VOLCANIC berdampingan dalam satu respons: record lama tanpa `confidence_level` dan record baru dengan `confidence_level`, berjarak 10 detik.

7 Problem 5: Fan-out Event Distribution dan Pub-sub

Pertanyaan Utama

Bagaimana HazardEvent baru didistribusikan ke banyak consumer tanpa producer memanggil consumer secara langsung atau terhambat ketika salah satu consumer mati?

7.1 Rancangan Solusi

Distribusi event memakai RabbitMQ 3.13 dengan satu exchange bertipe *fanout* bernama `hazard_events_exchange`. Producer (Aggregator) hanya bertanggung jawab atas exchange ini; setiap consumer mendeklarasikan dan mengikat queue durable miliknya sendiri (`dashboard_updater_queue` dan `notifier_queue`) lewat job sekali-jalan (`init-topology`) yang dijalankan sebelum Aggregator start. Setiap queue menerima salinan penuh dari setiap event, sehingga consumer tidak bersaing memperebutkan pesan, dan menambah consumer baru tidak pernah mengubah satu baris pun kode producer.

Penyimpanan dan publikasi event memakai pola *transactional outbox*. Saat Aggregator menyimpan HazardEvent baru ke Canonical Store, baris outbox (`message_id`, `correlation_id`, payload) ditulis pada transaksi database yang sama. Komponen `OutboxDispatcher` – berjalan sebagai *timer* asinkron di dalam proses Aggregator yang sama, terpisah dari alur polling, bukan proses sistem operasi lain – mengklaim baris outbox yang tertunda (`FOR UPDATE SKIP LOCKED`) setiap detik dan mempublikasikannya melalui *publisher* yang menggunakan confirm channel. Pesan dikirim dengan flag persisten dan dibungkus dalam envelope `HazardEventMessage` yang memuat `message_id`, `correlation_id`, dan `payload`. Jika publish gagal atau broker tidak tersedia, baris outbox dilepas dan dicoba ulang pada tick berikutnya;

klaim memakai sewa (*lease*) sehingga proses yang crash di tengah publikasi tidak mengunci baris itu selamanya. Karena penulisan event dan niat publikasinya komit bersama dalam satu transaksi, kegagalan broker pada saat penerbitan tidak pernah menghilangkan event secara diam-diam.

Setiap consumer menjalankan langkah yang sama. Ack manual dengan prefetch 1 membuat pesan diproses satu per satu. Sebelum side effect dijalankan, consumer memeriksa `payload.hazard_id` di set dedup. Jika sudah pernah diproses, pesan langsung di-ack. Jika side effect gagal, pesan di-requeue. Envelope yang rusak dibuang.

7.2 Alternatif dan Trade-off

Keputusan	Keuntungan	Trade-off dan mitigasi
RabbitMQ fanout (dipilih) dibanding Kafka	Model queue per consumer sederhana. Producer tidak tahu siapa consumer-nya. Cocok untuk skala M1.	Tidak ada log yang bisa diputar ulang oleh consumer baru. Event yang sudah di-ack tidak bisa diambil lagi.
Fanout (dipilih) dibanding topic atau consumer group	Setiap consumer menerima salinan penuh tanpa perlu routing key.	Consumer yang ingin subset harus memfilter sendiri.
At-least-once (dipilih) dibanding at-most-once (auto-ack)	Event tidak hilang saat consumer crash sebelum memproses. Lebih aman untuk peringatan evakuasi.	Duplikat mungkin terjadi. Ditangani dengan dedup dan idempotensi.
Dedup in-memory dengan batas 10.000 id (dipilih) dibanding dedup persisten	Tanpa dependensi tambahan. Cukup untuk M1.	Set hilang saat restart. Dijelaskan pada Batasan.
Pull dari broker (dipilih) dibanding HTTP push ke consumer	Consumer mengatur laju sendiri lewat prefetch. Producer tidak perlu tahu endpoint consumer.	Perlu broker tambahan yang harus selalu tersedia.

Tabel 11: Keputusan desain dan trade-off untuk distribusi event.

7.3 Batasan yang Disadari

1. **Dedup hilang saat restart.** Consumer yang crash tepat setelah memproses tetapi sebelum ack bisa menerima pesan yang sama lagi setelah restart. Pesan itu akan diproses dua kali. Semantik at-least-once tetap terjaga.
2. **Tidak ada dead-letter queue.** Pesan yang selalu gagal di side effect akan di-requeue berulang. Consumer tetap berjalan, tetapi pesan itu tidak pernah selesai.
3. **Kapasitas dedup terbatas.** Set menyimpan 10.000 hazard_id terakhir. Event yang lebih lama bisa diproses ulang jika muncul kembali sebagai duplikat.
4. **Outbox belum punya dead-letter setelah percobaan maksimum.** Baris outbox yang terus gagal dipublikasikan akan dicoba ulang tanpa batas interval mundur (*backoff*) eksponensial; tidak ada ambang jumlah percobaan yang memindahkan baris itu ke status

gagal permanen untuk investigasi manual.

7.4 Bukti Penyelesaian

Bukti berikut diambil dari pengujian langsung terhadap sistem yang berjalan penuh lewat `docker compose up`, dijalankan 2026-10-07, lengkap dengan `hazard_id` dan `correlation_id` asli dari log tiap service.

No	Skenario	Harapan	Hasil
1	Jalankan *dashboard-updater* dan *notifier*, publish satu event	Kedua consumer memproses event yang sama	**Sesuai.** `hazard_id 8c93bb08...` dan `correlation_id f0998e6a...` diproses kedua consumer pada detik yang sama (`09:06:24.511` dan `09:06:24.512`).
2	Hentikan *notifier*, publish 2 sampai 3 event	*dashboard-updater* tetap menerima, publisher tidak error	**Sesuai.** Selama 25 detik *notifier* mati, *dashboard-updater* tetap memproses 4 event baru (`hazard_id 4a73467b`, `929a2a56`, `0d2d37c0`, dan satu lagi) tanpa jeda atau error pada publisher.
3	Jalankan kembali *notifier*	Event yang tertunda tersusul dari queue	**Sesuai.** Begitu *notifier* tersambung kembali (`"message":"consuming"` pukul `09:06:54.303`), 6 event diproses beruntun dalam kurang dari 1 detik, termasuk 3 `hazard_id` yang sama persis dengan yang sudah diproses *dashboard-updater* saat *notifier* mati – bukti queue durable menahan pesan, bukan membuangnya. Lihat potongan log di bawah.
4	Jalankan *demo-logger* di topic yang sama	Menerima event baru tanpa perubahan producer	**Sesuai.** *demo-logger* langsung mencatat `bound, waiting for events`, lalu menerima 3 event baru (`hazard_id c26f8bbc`, `32f2a936`, `dc4737f5`) lengkap dengan `correlation_id` masing-masing, tanpa satu baris pun diubah di `event-publisher.ts`.

Tabel 12: Skenario bukti Problem 5, diuji 2026-10-07.

```
# Potongan log notifier tepat setelah tersambung kembali (skenario 3):
{"service":"notifier","queue":"notifier_queue","message":"consuming"}
{"service":"notifier","correlation_id":"3468dd8a-...","hazard_id":"4a73467b-...",
 "message":"notification dispatched for hazard event"}
{"service":"notifier","correlation_id":"56b6f062-...","hazard_id":"929a2a56-...",
 "message":"notification dispatched for hazard event"}
{"service":"notifier","correlation_id":"04196d74-...","hazard_id":"0d2d37c0-...",
 "message":"notification dispatched for hazard event"}
# Ketiga hazard_id di atas sama persis dengan yang sudah diproses dashboard-updater
# saat notifier mati, dan tersusul kurang dari 1 detik setelah notifier menyala.
```

Verifikasi otomatis: 14 unit test untuk setiap consumer lulus, termasuk dedup dan requeue.

Pembaruan 2026-10-09: transactional outbox. Setelah pengujian langsung di atas, mekanisme publikasi diperkuat dengan pola *transactional outbox* (lihat Rancangan Solusi) untuk menutup celah: sebelumnya, jika Aggregator berhenti tepat setelah event tersimpan di Canonical Store tetapi sebelum `publish()` ke broker selesai, event itu tidak pernah terkirim. Perubahan ini belum diuji ulang lewat `docker compose` langsung (hanya diverifikasi lewat unit test di bawah ini, dijalankan 2026-10-09), sehingga klaimnya dibatasi pada apa yang benar-benar diverifikasi:

- `outbox-dispatcher.test.ts`: dua test mencakup pemulihan sebuah publikasi yang gagal dikonfirmasi lalu berhasil pada percobaan berikut dengan `message_id` yang sama persis (membuktikan proses yang di-restart mengambil state dari baris outbox yang persisten, bukan dari memori), serta baris yang sudah terkonfirmasi tidak dipublikasikan ulang pada *flush* berikutnya.
- `outbox-transaction.test.ts`: memverifikasi baris outbox hanya ditulis saat `upsert` benar-benar membuat baris baru (`created = true`), dan penulisan event plus baris outbox terjadi dalam satu transaksi database yang sama.

7.5 Jawaban atas Butir "Di Laporan"

1. **Broker dan pola.** RabbitMQ dengan exchange fanout dan queue durable per consumer.
2. **Semantik pengiriman.** At-least-once. Pesan dipersistenkan oleh publisher dan di-ack manual oleh consumer setelah side effect berhasil.
3. **Konsekuensi bagi consumer.** Redelivery adalah kondisi normal, sehingga consumer wajib idempoten. Setiap consumer memeriksa `hazard_id` sebelum side effect. Kelompok menyikapinya dengan dedup in-memory untuk M1 dan mencatat batasannya.
4. **Consumer mati.** Producer tidak tertahan dan consumer lain tetap menerima. Event yang terbit selama consumer mati tersimpan di queue durable dan tersusul setelah consumer dijalankan kembali, selama broker tetap hidup.
5. **Consumer baru.** Consumer baru cukup mendeklarasikan queue sendiri dan mengikat ke exchange. Kode producer tidak perlu diubah sama sekali – producer hanya memiliki `hazard_events_exchange` dan tidak menyimpan daftar queue consumer mana pun. Dibuktikan oleh `tests/audit/p5-third-consumer.cjs`: consumer audit independen yang mengikat queue sementara ke exchange yang sama dan menerima event tanpa mengimpor atau mengubah kode producer.

A Bukti Pengujian Tambahan